

File Class in C# with Examples

In this article, I will discuss **How to Implement File Handling using File Class in C#** with Examples. Please read our previous article discussing [How to Implement File Handling in C# using StreamWriter and StreamReader](#) class with Examples.

File Class in C#

The File Class in C# Provides some static methods to perform most file operations, such as Creating a File, Copying and Moving a File, Deleting Files, and working with FileStream to read and write streams. The **File** class is defined in the **System.IO** namespace. There can be situations where you want to work with files directly. The basic file operations that we generally do are as follows:

1. **Reading:** This operation is the basic Read Operation wherein data is read from a file.
2. **Writing:** This operation is the basic Write Operation wherein data is written to a file. All existing contents are removed from the file by default, and new content is written.
3. **Appending:** This operation also involves writing information to a file. The only difference is that the existing data in a file is not overwritten. The new data to be written is added at the end of the file.

The File class in C# exposes many static methods for moving, copying, reading, writing, and deleting files. The File belongs to the System.IO namespace; if you go to the definition of the File class, you will see the following.

```
namespace System.IO
{
    ...public static class File
    {
        ...public static void AppendAllLines(string path, IEnumerable<string> contents);
        ...public static void AppendAllLines(string path, IEnumerable<string> contents, Encoding encoding);
        ...public static void AppendAllText(string path, string contents, Encoding encoding);
        ...public static void AppendAllText(string path, string contents);
        ...public static StreamWriter AppendText(string path);
        ...public static void Copy(string sourceFileName, string destFileName);
        ...public static void Copy(string sourceFileName, string destFileName, bool overwrite);
        ...public static FileStream Create(string path);
        ...public static FileStream Create(string path, int bufferSize);
        ...public static FileStream Create(string path, int bufferSize, FileOptions options);
        ...public static FileStream Create(string path, int bufferSize, FileOptions options, FileSecurity fileSecurity);
        ...public static StreamWriter CreateText(string path);
        ...public static void Decrypt(string path);
        ...public static void Delete(string path);
        ...public static void Encrypt(string path);
        ...public static bool Exists(string path);
        ...public static FileSecurity GetAccessControl(string path);
        ...public static FileSecurity GetAccessControl(string path, AccessControlSections includeSections);
        ...public static FileAttributes GetAttributes(string path);
        ...public static DateTime GetCreationTime(string path);
        ...public static DateTime GetCreationTimeUtc(string path);
        ...public static DateTime GetLastAccessTime(string path);
        ...public static DateTime GetLastAccessTimeUtc(string path);
        ...public static DateTime GetLastWriteTime(string path);
        ...public static DateTime GetLastWriteTimeUtc(string path);
        ...public static void Move(string sourceFileName, string destFileName);
        ...public static FileStream Open(string path, FileMode mode, FileAccess access, FileShare share);
        ...public static FileStream Open(string path, FileMode mode);
        ...public static FileStream Open(string path, FileMode mode, FileAccess access);
        ...public static FileStream OpenRead(string path);
    }
}
```

File Class Methods in C#:

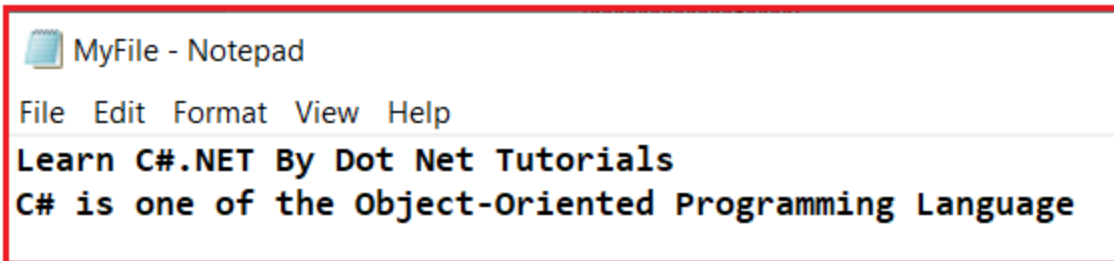
The following are commonly used methods of File class in C#.

1. **Copy:** This method is used to Copy an existing file to a new file. Overwriting a file of the same name is not allowed.
2. **Create:** This method creates or overwrites it in the specified path.
3. **Decrypt:** This method is used to Decrypt a file encrypted by the current account using the `System.IO.File.Encrypt(System.String)` method.
4. **Delete:** This method is used to delete the specified file.
5. **Encrypt:** This method is used to encrypt a file so that only the account used to encrypt the file can decrypt it.
6. **Open:** This method is used to Open a `System.IO.FileStream` on the specified path, having the specified mode with reading, write, or read/write access and the specified sharing option.
7. **Move:** This method is used to Move a specified file to a new location, providing the option to specify a new file name.
8. **Exists:** This method is used to determine whether the specified file exists.
9. **OpenRead:** This method is used to open an existing file for reading.
10. **OpenText:** This method opens an existing UTF-8 encoded text file for reading.
11. **OpenWrite:** This method is used to open an existing file or create a new file for writing.
12. **ReadAllBytes:** This method is used to open a binary file, read the file's contents into a byte array, and then close the file.
13. **ReadAllLines:** This method is used to Open a file, read all lines of the file with the specified encoding, and then close the file.
14. **ReadAllText:** This method is used to Open a text file, read all the text in the file, and then close the file.
15. **ReadLines:** This method is used to read the lines of a file.
16. **Replace:** This method is used to replace a specified file's contents with another file's contents, delete the original file, and create a backup of the replaced file.
17. **WriteAllBytes:** This method is used to create a new file, write the specified byte array to the file, and then close the file. If the target file already exists, it is overwritten.
18. **WriteAllLines:** This method is used to create a new file, write the specified string array to the file, and then close the file.
19. **WriteAllText:** This method is used to create a new file, write the specified string to the file, and then close the file. If the target file already exists, it is overwritten.

Example to Understand File Class in C#:

The C# Language and .NET Framework can work with files with the help of several methods of File class. Let us see how to use the File class static methods to perform different file operations with examples. Let us assume we have a file in the **D drive** called **MyFile.txt**. The **MyFile.txt** file will be a simple text file and have 2 lines of data as follows:

[Learn C#.NET By Dot Net Tutorials](#)
[C# is one of the Object-Oriented Programming Language](#)



Now, we will create a simple Console Application and work with File class static methods to perform different file operations.

Exists Method of File Class in C#

The Exists Method of File Class in C# is used to check whether a particular file exists. This method will return true if the caller has the required permissions and the path contains the name of an existing file; otherwise, it will return false. This method also returns false if the **Path is null**, an **Invalid Path**, or a **Zero-Length string**. If the caller has insufficient permissions to read the specified file, no exception is thrown, and the method returns false regardless of the path's existence.

Now, let's see the code that can be used to check whether our MyFile.txt file exists. So, copy and paste the code below into your Console Application's Program class. The Exists Method is a static method of the File class, so we call this method with the class name, i.e., File. This method expects one parameter, which is the file path.

```
using System;
using System.IO;
namespace FileClassDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Set the File Path
            string FilePath = @"D:\MyFile.txt";
            //Check if the File Exists using the Static Exists Method of the File Class
            //Exists Method will take the string File Path as a parameter
            if (File.Exists(FilePath))
            {
                Console.WriteLine("MyFile.txt File Exists in D Directory");
            }
            else
            {
                Console.WriteLine("MyFile.txt File Does Not Exist in D Directory");
            }
            Console.ReadKey();
        }
    }
}
```

Code Explanation:

First, we are storing the **MyFile.txt** file path in a string variable called `FilePath`. Then, we use the `Exists` method to check whether the file exists. A true value will be returned if the File exists in the specified path.

If we get a true value, then we write the message “**MyFile.txt File Exists in D Directory**” to the console window; else, If we get a false value, then we write the message “**MyFile.txt File Does Not Exist in D Directory**” to the console window. So, when you run the above code, you will get the following output as the file `MyFile.txt` Exists in the D Directory.

```
MyFile.txt File Exists in D Directory
```

ReadAllLines Method of File Class in C#:

The `ReadAllLines` Static Method of `File` Class in C# is used to open a file, read all the lines one by one from the file, and then close the file. The lines are then stored in a string array variable.

For a better understanding, please have a look at the below example. In the below example, first, we are creating a string variable to store the file path. Then, we check whether the file exists using the static `Exists` method of the `File` class. If the file exists, we read all the lines from it using the static `ReadAllLines` method of the `File` class and storing it in a string array. Then, we use a for each loop to read all the lines one by one and print the same on the console window. The following code is self-explained, so please go through the comment lines for a better understanding.

```
using System;
using System.IO;
namespace FileClassDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Set the File Path
            string FilePath = @"D:\MyFile.txt";
            //Check if the File Exists using the Static Exists Method of the File Class
            //Exists Method will take the string File Path as a parameter
            if (File.Exists(FilePath))
            {
                //If the File Exists, Read All the Lines from the File using the ReadAllLines
                //ReadAllLines Method returns a string array
                string[] lines = File.ReadAllLines(FilePath);
                //Using a for Each loop to access each line from the string array
                foreach (var line in lines)
                {
                    Console.WriteLine(line);
                }
            }
        }
    }
}
```

```

        else
        {
            Console.WriteLine("MyFile.txt File Does Not Exist in D Directo
        }
        Console.ReadKey();
    }
}
}

```

As our file contains two lines of data, when you run the above code, you will get the following output.

```

Learn C#.NET By Dot Net Tutorials
C# is one of the Object-Oriented Programming Language

```

ReadAllText Method of File Class in C#

The ReadAllText Method of File Class in C# is used to open a text file, read all the text in the file, and then close the file. In this case, it will read all the text from the file simultaneously. The text is then stored in a string variable.

For a better understanding, please have a look at the below example. In the below example, first, we are creating a string variable called the FilePath to store the file path. Then, we check whether the file exists using the Static Exists method of File Class. If the file exists, then we are reading all the text from the file at once by using the ReadAllText method of the File class and storing the result in a string variable. Then, we print the same on the console window. The following code is self-explained, so please go through the comment lines for a better understanding.

```

using System;
using System.IO;
namespace FileClassDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Set the File Path
            string FilePath = @"D:\MyFile.txt";
            //Check if the File Exists using the Static Exists Method of the F
            //Exists Method will take the string File Path as a parameter
            if (File.Exists(FilePath))
            {
                //If the File Exists, Read All Text from a file at once using
                //ReadAllText: Opens a text file, reads all the text in the fi
                then closes the file.
                //ReadAllText Method returns a string containing all the text
                file.
                string lines = File.ReadAllText(FilePath);
                Console.WriteLine(lines);
            }
        }
    }
}

```

```

    }
    else
    {
        Console.WriteLine("MyFile.txt File Does Not Exist in D Directo
    }
    Console.ReadKey();
}
}
}
}

```

When you run the above code, you will get the following output.

```

Learn C#.NET By Dot Net Tutorials
C# is one of the Object-Oriented Programming Language

```

Copy Method of File Class in C#:

The static Copy Method of File Class in C# is used to make a copy of an existing file. The most important point you need to remember is that overwriting a file of the same name is not allowed using the Copy method of the File class. You need to give a different name. The Copy method takes two parameters. The first parameter is the sourceFileName, i.e., the file to copy, and the second parameter is the destFileName, i.e., the name of the destination file, and the destination file cannot be a directory or an existing file.

For a better understanding, please have a look at the below example. In the below example, first, we create two string variables called SourceFilePath and DestinationFilePath to store the source and destination file paths, respectively. Then, we check whether the source file exists using the static Exists method of the File class. If the source file exists, we call the static Copy method of the File class to copy the source file **MyFile.txt** file to the destination file **MyFileCopy.txt**. Then, we print the destination file data on the console window using the ReadAllText static method of the File class. The following code is self-explained, so please go through the comment lines for a better understanding.

```

using System;
using System.IO;
namespace FileClassDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Set the Source and Destination File Paths
            string SourceFilePath = @"D:\MyFile.txt";
            string DestinationFilePath = @"D:\MyFileCopy.txt";
            //Check if the SourceFilePath Exists using the Exists Method of the
Class
            if (File.Exists(SourceFilePath))
            {
                //If the SourceFilePath Exists, then copy the File content to
using the Copy Method
                //We are copying the SourceFilePath to DestinationFilePath
                File.Copy(SourceFilePath, DestinationFilePath);
            }
        }
    }
}

```

```

        //Read the Text from DestinationFilePath using ReadAllText Method
        string lines = File.ReadAllText(DestinationFilePath);
        //Print the Data of the DestinationFilePath
        Console.WriteLine(lines);
    }
    else
    {
        Console.WriteLine("MyFile.txt File Does Not Exist in D Directory");
    }
    Console.ReadKey();
}
}
}
}
}

```

So, when you run the above code, you will get the following output.

```

Learn C#.NET By Dot Net Tutorials
C# is one of the Object-Oriented Programming Language

```

Now you can see that **MyFileCopy.txt** should be created inside the D drive. You need to remember that the destination file cannot be a directory or an existing file. For example, the **MyFileCopy.txt** file is already created inside the D drive. Again, if you run the same code, you will get the following exception.

System.IO.IOException: 'The file 'D:\MyFileCopy.txt' already exists.'

Another overloaded version of the Copy method is available inside the File class with the following signature. To override an existing file, you can pass the third parameter as true or false. So, the following overloaded version of the Copy method is used to copy an existing file to a new file. Overwriting a file of the same name is allowed.

public static void Copy(string sourceFileName, string destFileName, bool overwrite);

So, let us modify the previous example, use the above-overloaded version, and see the output.

```

using System;
using System.IO;
namespace FileClassDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Set the Source and Destination File Paths
            string SourceFilePath = @"D:\MyFile.txt";
            string DestinationFilePath = @"D:\MyFileCopy.txt";
            //Check if the SourceFilePath Exists using the Exists Method of the
            Class
            if (File.Exists(SourceFilePath))
            {

```

```

        //If the SourceFilePath Exists, then copy the File content to
using the Copy Method
        //We are coping the SourceFilePath to DestinationFilePath
        //Passing the third parameter boolean overwrite as true
        File.Copy(SourceFilePath, DestinationFilePath, true);
        //Read the Text from DestinationFilePath using ReadAllText Met
        string lines = File.ReadAllText(DestinationFilePath);
        //Print the Data of the DestinationFilePath
        Console.WriteLine(lines);
    }
    else
    {
        Console.WriteLine("MyFile.txt File Does Not Exist in D Directo
    }
    Console.ReadKey();
}
}
}
}

```

Now, run the above code, and you will not get any errors.

Delete Method of File Class in C#:

The static Delete method of the File Class in C# is used to delete an existing file. The Delete method takes one parameter, which is the path of the file to be deleted.

For a better understanding, please have a look at the below example. In the below example, first, we are creating a string variable called FilePath to store the file path. Then, we check whether the file exists using the Exists method of the File class. If the file exists, then we call the static Delete method of the File class, passing the file path to delete the **MyFileCopy.txt** file. The following code is self-explained, so please go through the comment lines for a better understanding.

```

using System;
using System.IO;
namespace FileClassDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Set the File Path
            string FilePath = @"D:\MyFileCopy.txt";
            //Check if the FilePath Exists using the Exists Method of the File
            if (File.Exists(FilePath))
            {
                //If the FilePath Exists, then Delete the File using the Delet
the File class
                //To the Delete Method, pass the FilePath which File you want
                File.Delete(FilePath);
                Console.WriteLine("MyFileCopy.txt File Deleted");
            }
        }
    }
}

```

```

    }
    else
    {
        Console.WriteLine("MyFileCopy.txt File Does Not Exist in D Dir");
    }
    Console.ReadKey();
}
}
}

```

So, when you run the above code, you will get the following output.

MyFileCopy.txt File Deleted

Create Method of File Class in C#

The static Create Method of File Class in C# creates a file in the specified folder or directory. There are multiple overloaded versions of this method available in the File class. They are as follows:

1. **public static FileStream Create(string path)**: Creates or overwrites a file in the specified path.
2. **public static FileStream Create(string path, int bufferSize)**: Creates or overwrites the specified file. The bufferSize parameter specifies the number of bytes buffered for reads and writes to the file.
3. **public static FileStream Create(string path, int bufferSize, FileOptions options)**: Creates or overwrites the specified file, specifying a buffer size and a FileOptions value that describes how to create or overwrite the file.
4. **public static FileStream Create(string path, int bufferSize, FileOptions options, FileSecurity fileSecurity)**: Creates or overwrites the specified file with the specified buffer size, file options, and file security.

Note: All the above-overloaded versions return an instance of the FileStream class. So, we need to close the stream object by calling the Close method.

Let us understand the static Create method of the File class with an example. In the example below, first, we create a string variable called FilePath, which specifies the path of the File that we will create using the Create method. Then, we call the Create method by passing the FilePath, which will create the **MyNewFile.txt** file in the **D drive** and return the FileStream Object. Then, immediately, we close the **FileStream** Object by calling the **Close** method. Then, we check whether the **MyNewFile.txt** file exists using the Exists method of the File class. If the MyNewFile.txt file exists, then we are creating a string array that contains some data, and then we are calling the **WriteAllLines** method by passing the FilePath and string array. The WriteAllLines method will write the string array data into the specified file. And finally, we are reading the content of the MyNewFile.txt file using the ReadAllText method and printing it on the Console. The following code is self-explained, so please go through the comment lines for a better understanding.

```

using System;
using System.IO;
namespace FileClassDemo
{
    class Program
    {
        static void Main(string[] args)
        {

```

```

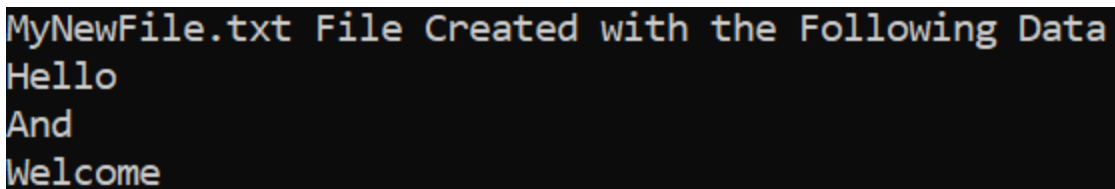
//Set the File Path where you want to create the File
string FilePath = @"D:\MyNewFile.txt";
//Creating the File using the static Create method of the File class
//To the Create Method, we need to pass the FilePath where we want
the File

//The Create Method will return the FileStream object
FileStream fileStream = File.Create(FilePath);
//Closing the FileStream Object immediately once the File is Created
fileStream.Close();

//Check if the FilePath Exists using the Exists Method of the File class
if (File.Exists(FilePath))
{
    //If the File Exists, Write Content to the File
    //Creating a string array to Hold the Data
    string[] content = { "Hello", "And", "Welcome" };
    //using the WriteAllLines Method, we are writing the data
    File.WriteAllLines(FilePath, content);
    Console.WriteLine("MyNewFile.txt File Created with the Following Data");
    //Reading the data from the File using ReadAllText Method
    string fileContent = File.ReadAllText(FilePath);
    //Printing the Data in the Console
    Console.WriteLine(fileContent);
}
else
{
    Console.WriteLine("MyNewFile.txt File Does Not Exist in D Drive");
}
Console.ReadKey();
}
}
}

```

So, when you run the above code, you will get the following output.



```

MyNewFile.txt File Created with the Following Data
Hello
And
Welcome

```

In the next article, I will discuss [TextWriter and TextReader Classes in C#](#) with Examples. Here, in this article, I try to explain **How to Implement File Handling using File Class in C#** with Examples. I hope you enjoy this File Handling using File Class in C# with Examples article. I would like to have your feedback. Please post your feedback, questions, or comments about this article.